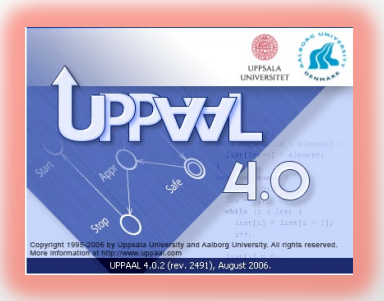
Formal Modeling and Verification



Muhammad Abdul Basit Ur Rahim, PhD

Objectives

- Learning the software modeling
- Know how to analyze a computational model.

Introduction

- Why modeling and verification of a system design is important?
  - It identifies the design error in earlier design phase.
- An error in later phases increases maintenance cost and chance of failure of product.
- Examples:



Fuel Filling Station



Chemical Mixing Plant



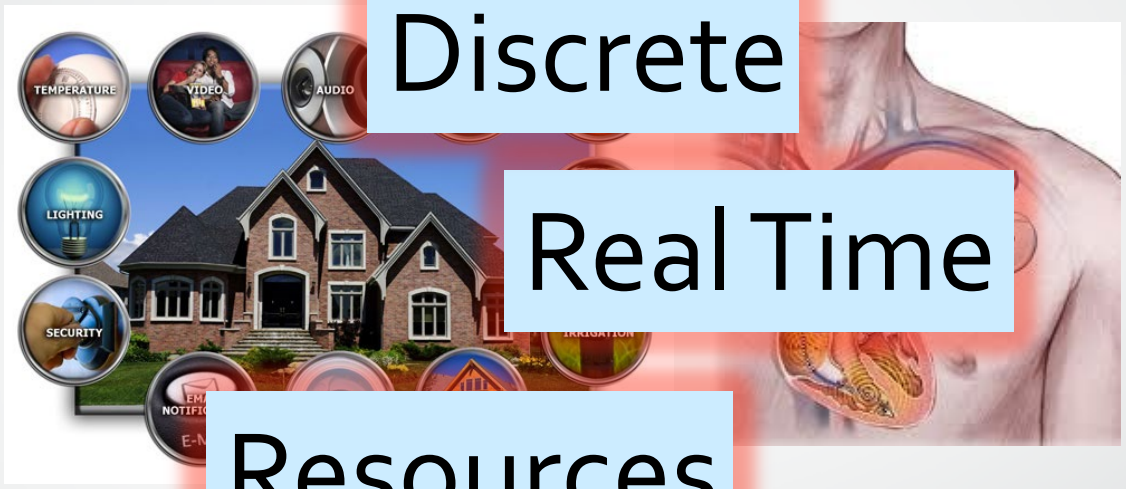
Flight System



Security System

From Computer Science to Cyber Physical Systems





Discrete

Real Time

Resources

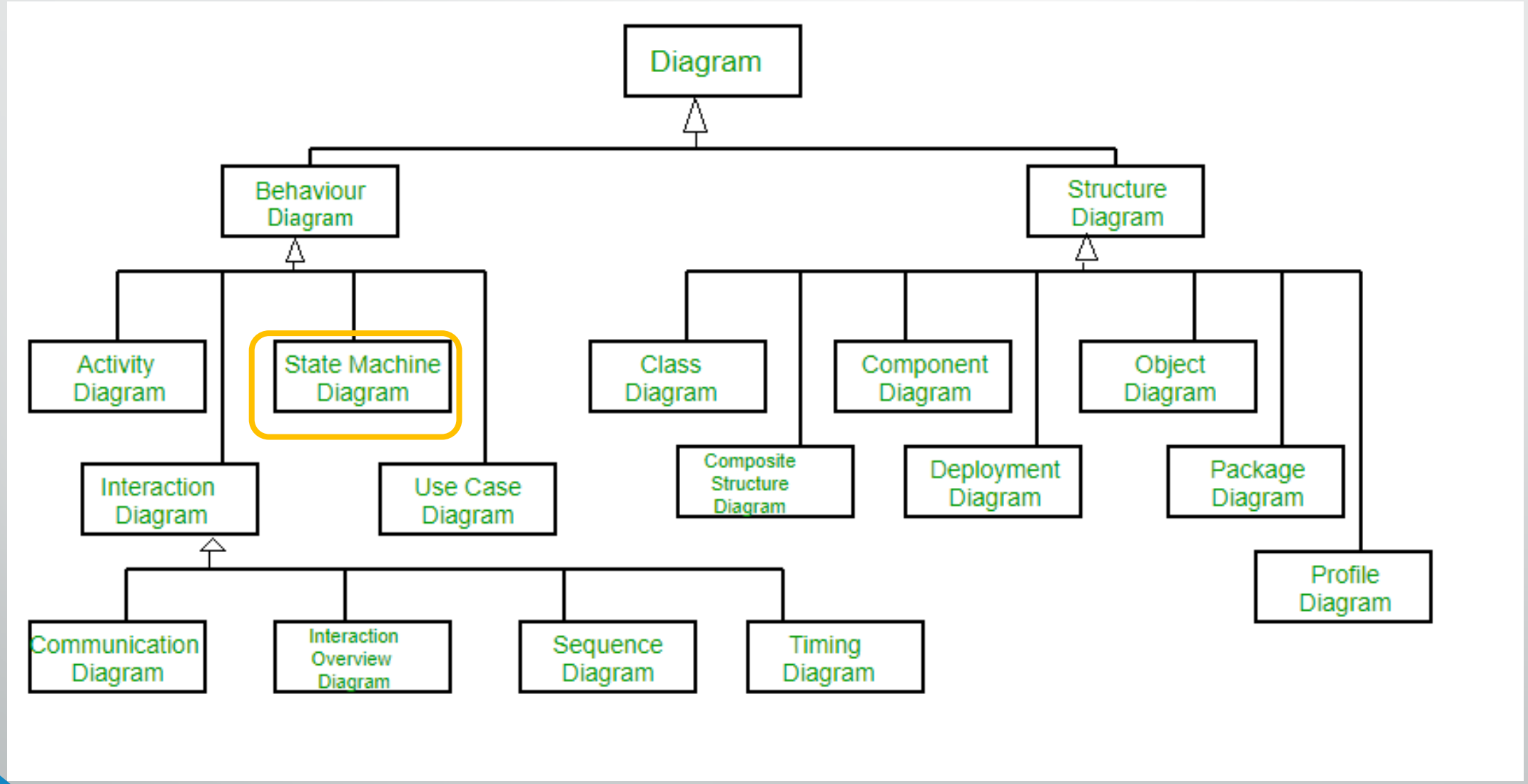




Stochasticity

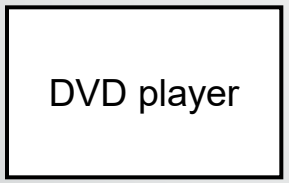
Hybrid

How to model a system

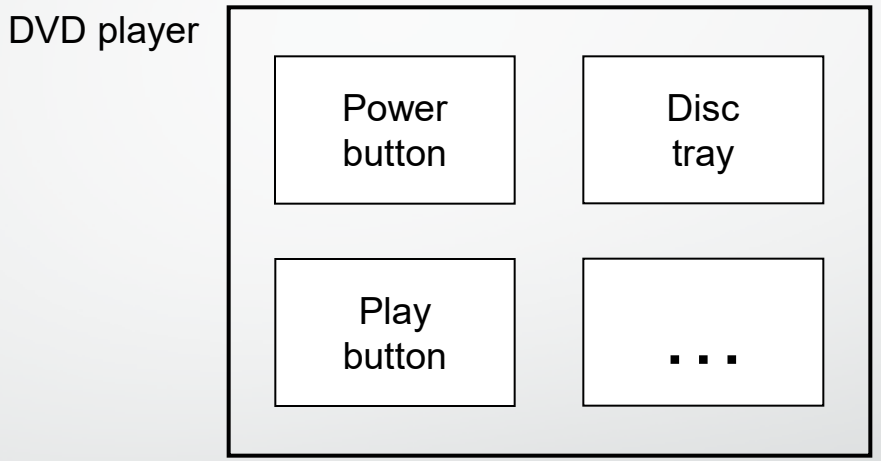


Different Abstractions

(Level of detail)



Atomic object

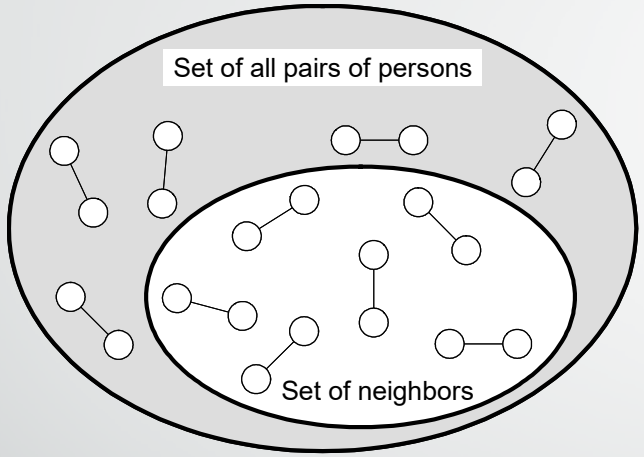


Object composed of parts

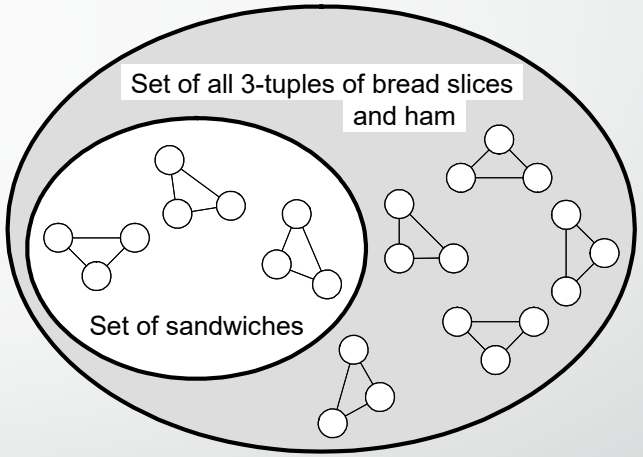
Example: States of a DVD Player

System Part (Object)	State Relations
Power button	{Off, On}
Disc tray	{Empty, Loaded}
Play button	{Off, On}
…	…

Relations: Examples



Relation: ***Neighbors*** (Person_i, Person_j)

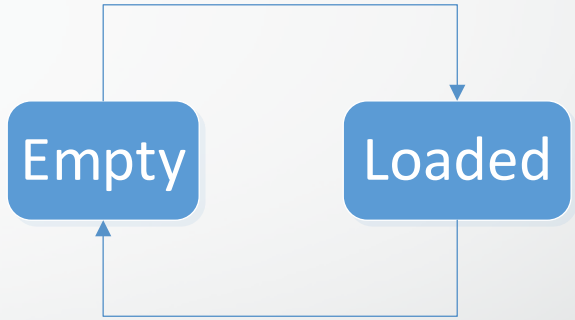


Relation:
Sandwich (Bread-slice, Ham-slice, Bread-slice)

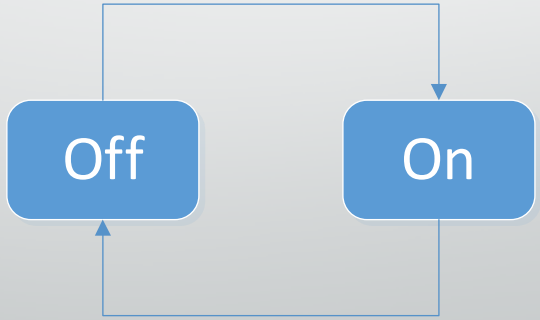
State Machine Diagram – DVD Player



Power Button



Disc Tray



Play Button

Two Types of Verification Procedure

1. Static verification (Analysis)

Inspecting the requirements before code runs.

- *Code conventions verification*
- *Bad practices detection*
- ***Formal verification using model checking tool***

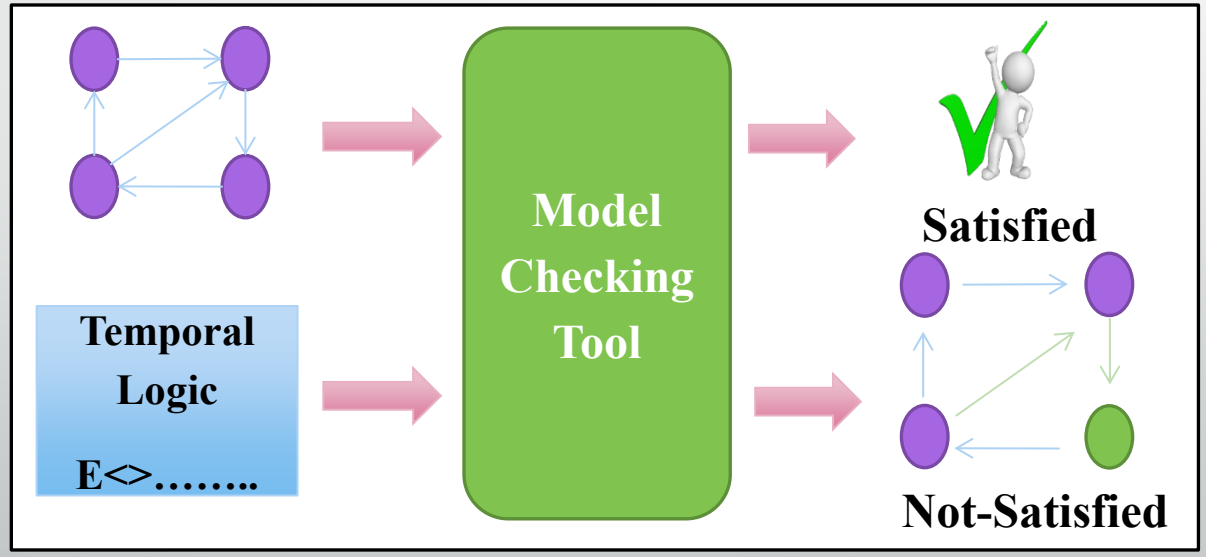
2. Dynamic verification (Test Phase)

Inspecting the requirements after the code runs.

- Test in the small: (Unit test)
- Test in the large: Acceptance test
- Functional test
- Non-functional test (performance, stress test)

Model Checking Tool

- Model checking or property checking is a method for checking whether a finite-state model of a system meets a given specification (a.k.a. correctness).

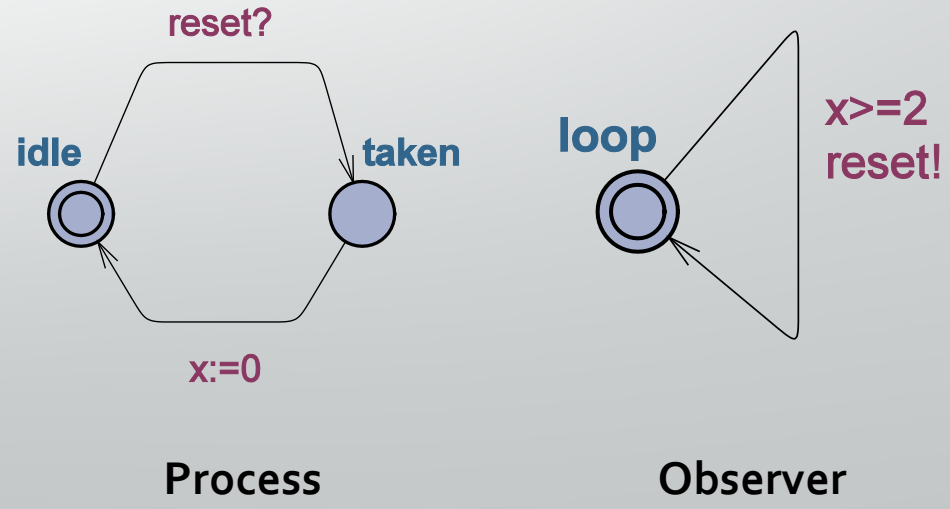


UPPAAL??!!

- **SWE**den + **DEN**mark = SWEDEN
  - REJECTED
- swe**DEN** + den**MARK** = DENMARK
  - REJECTED
- **UPP**sala + **AAL**borg = UPPAAL
  - ACCEPTED

UPPAAL – A Model Checking Tool

- A tool for **modeling**, **simulation** and **verification** of real-time systems.
- Appropriate for systems that can be modeled as
  - a collection of **non-deterministic processes** with **finite control structure** and **real-valued clocks** (i.e. timed automata)
- Typical application areas:
  - Real-time controllers
  - Communication protocols
  - Other systems in which timing aspects are critical
- Modeling language
  - channels, and locations
  - templates with local clocks
  - C subset



Hidden States



Observable state:
apple's appearance



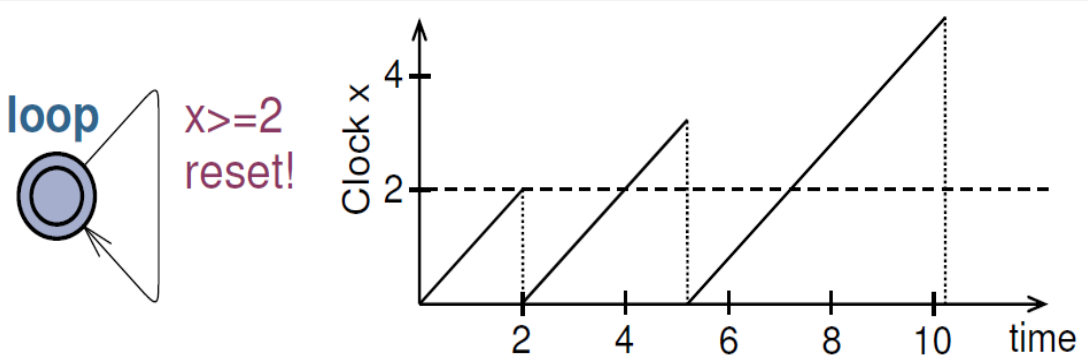
Hidden state:
contains a worm

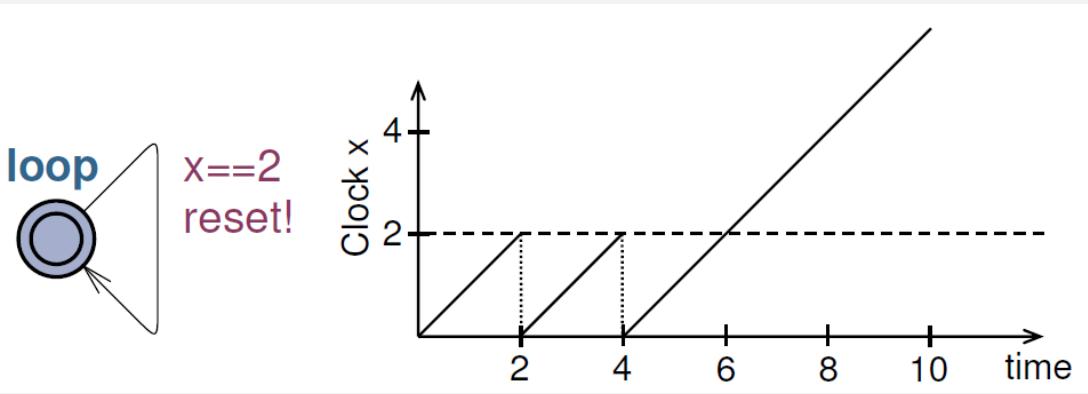


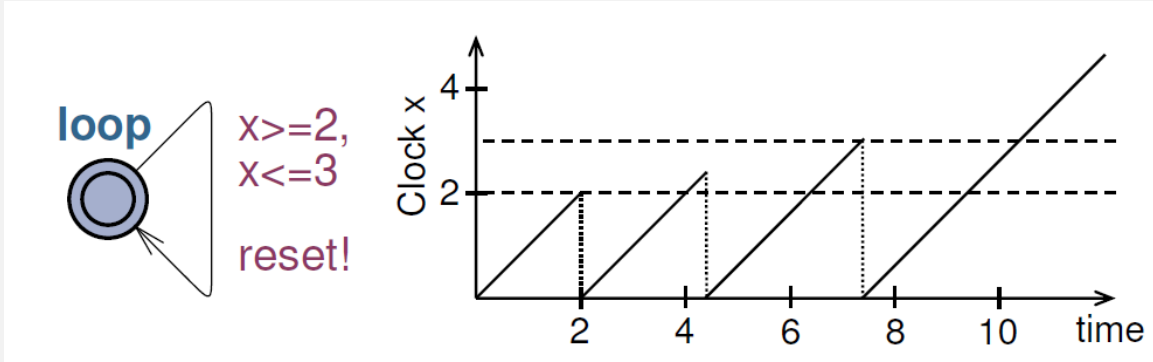
Goal:

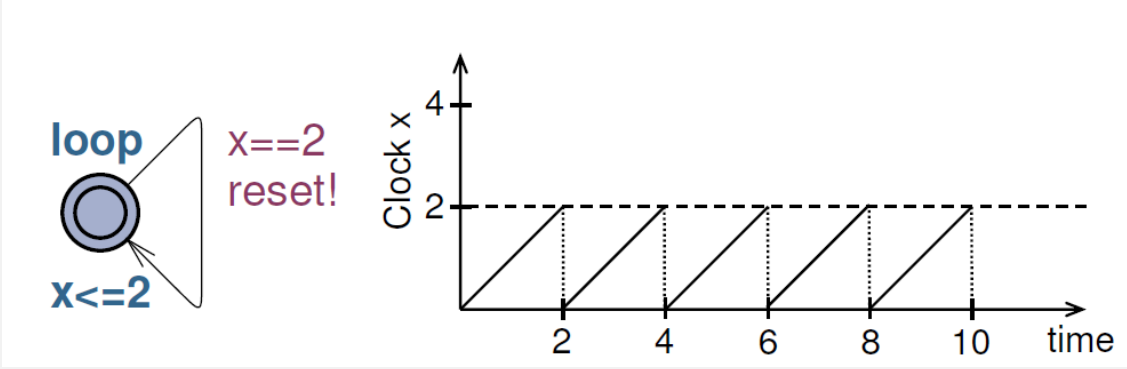
Find the likelihood of different hidden states, for given observable states

Delay Transition and Invariants





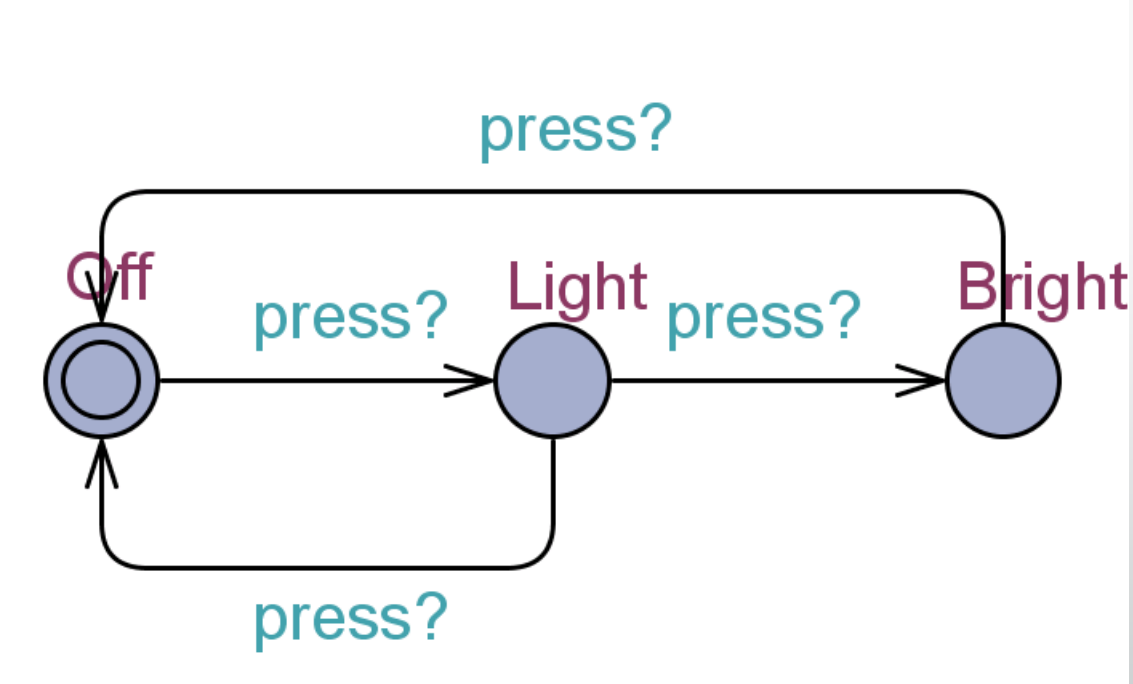




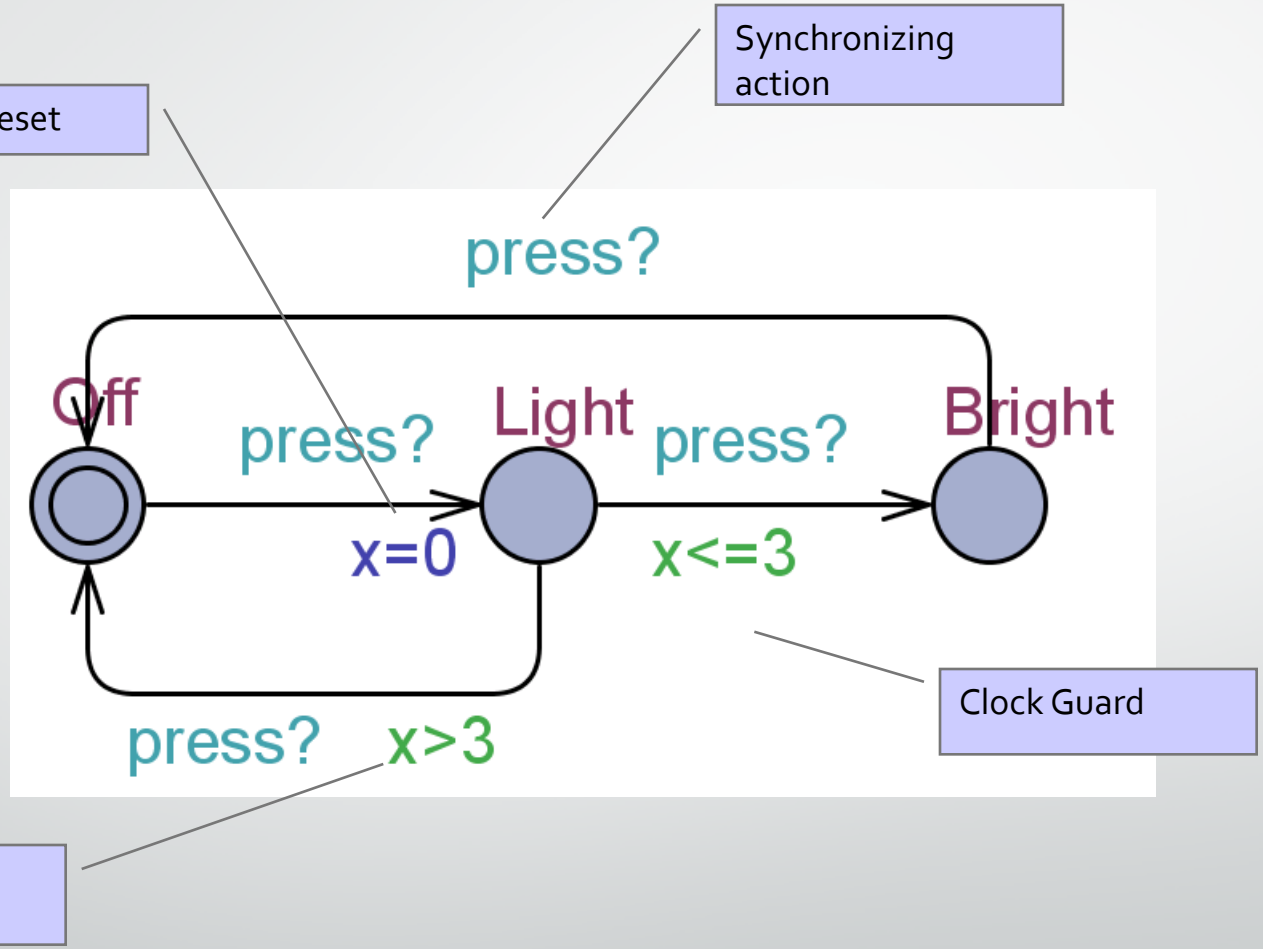
DEMO

Dumb Light

A Dumb Light Controller

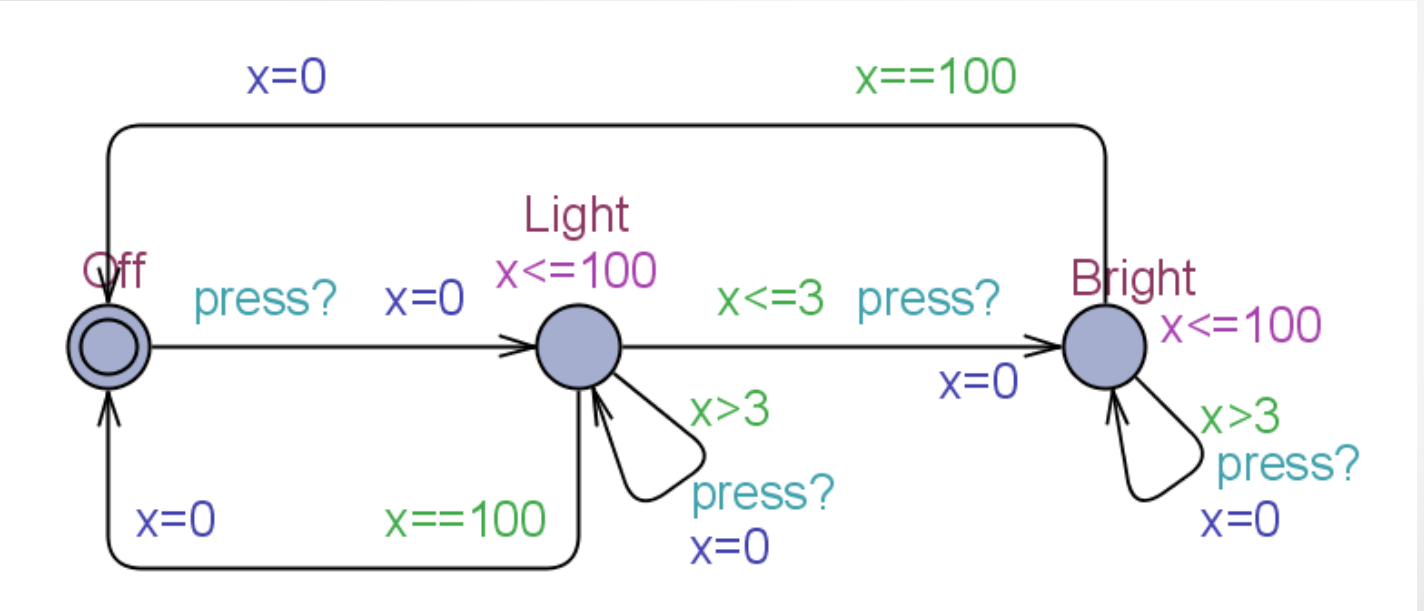


Intelligent Light Controller



ADD a clock x

Intelligent Light Controller



Transitions:

	(Off , x=0)
delay 4.32	→ (Off , x=4.32)
press?	→ (Light , x=0)
delay 4.51	→ (Light , x=4.51)
press?	→ (Light , x=0)
delay 100	→ (Light , x=100)
τ	→ (Off , x=0)

Note:

(Light , x=0) delay 103 →

X

Invariants ensures progress

DEMO

Train Crossing

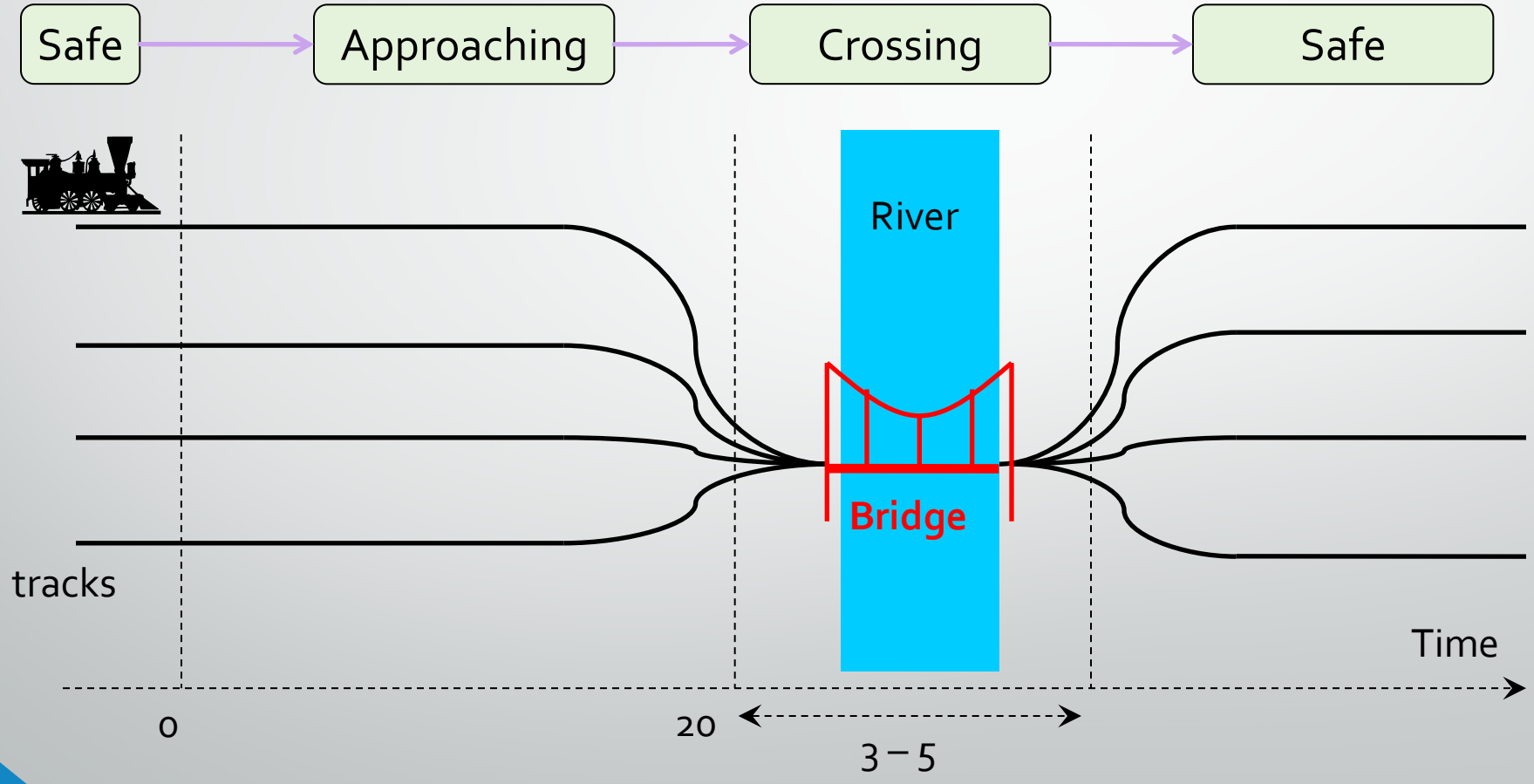
Train Crossing

The train gate example is distributed with UPPAAL.

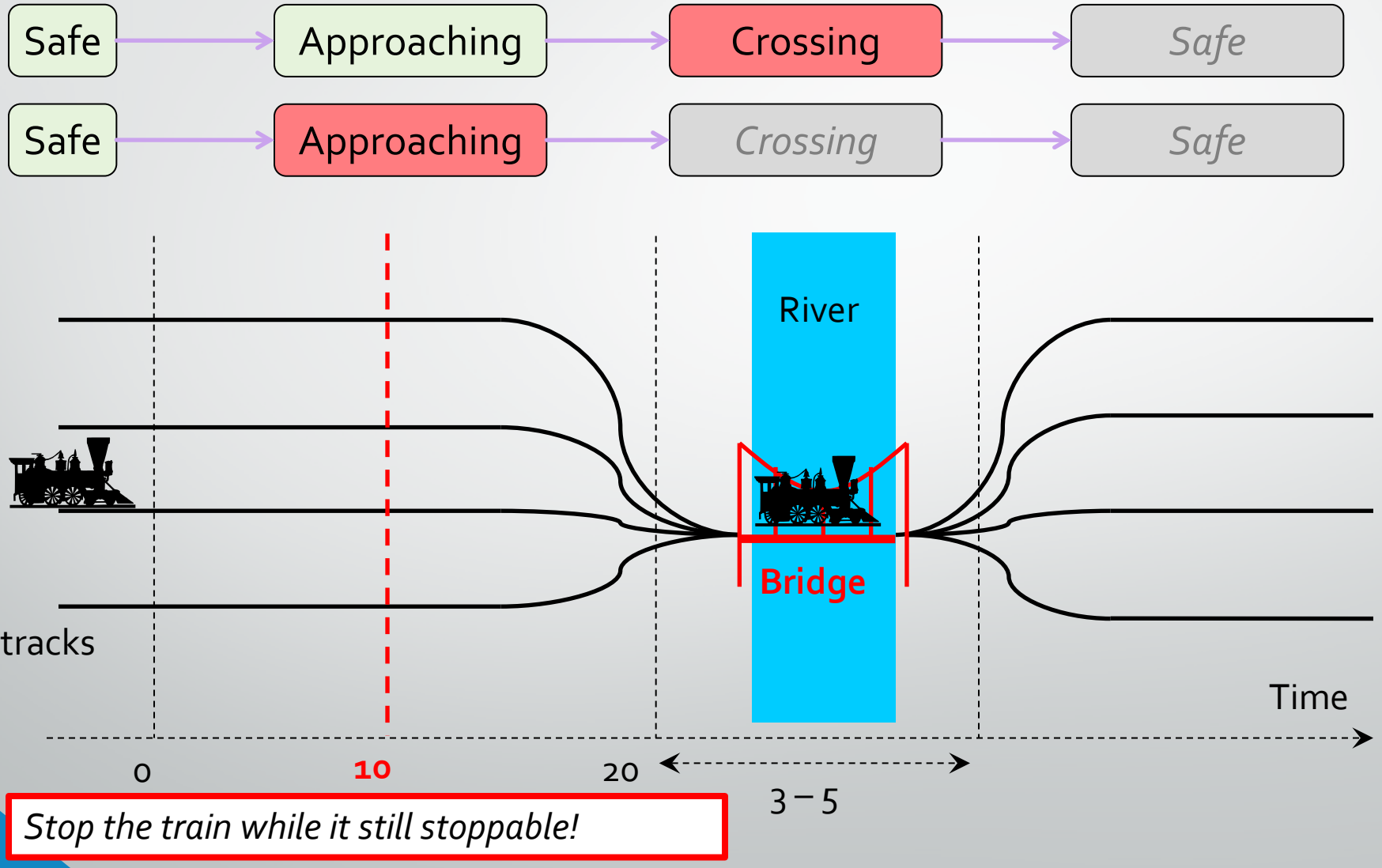
- It is a railway control system that controls access to a bridge for several trains.
- The bridge is a critical shared resource that may be accessed only by one train at a time.
- The system is defined as a number of trains (assume 4 for this example) and a controller.

- A train can not be stopped instantly and restarting also takes time. Therefore, there are timing constraints on the trains before entering the bridge.
- When approaching, a train sends an appr! signal. Thereafter, it has 10-time units to receive a stop signal. This allows it to stop safely before the bridge. After these 10 time units, it takes further 10 time units to reach the bridge if the train is not stopped.
- If a train is stopped, it resumes its course when the controller sends a go! signal to it after a previous train has left the bridge and sent a leave! signal.

Train Crossing

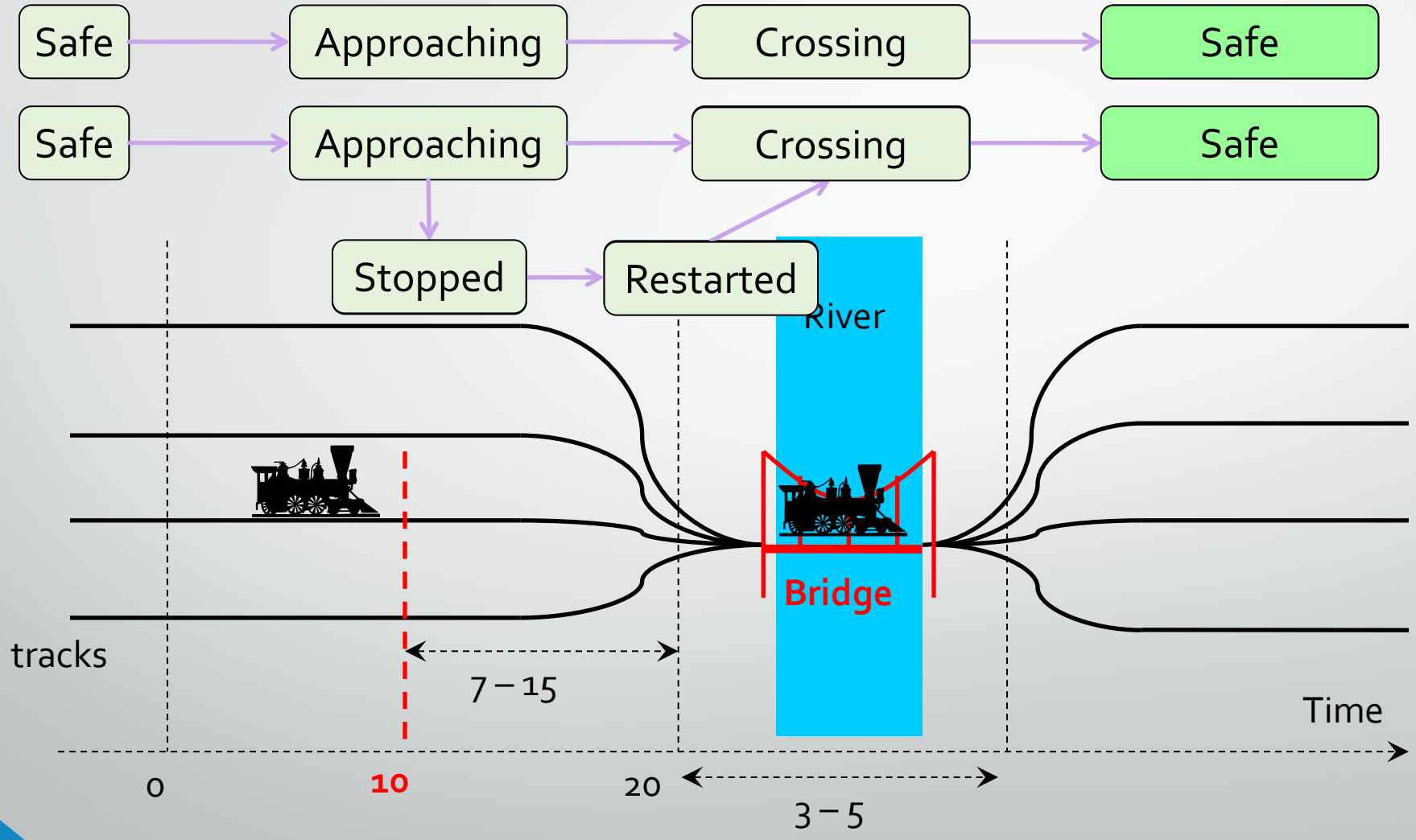


Train Crossing

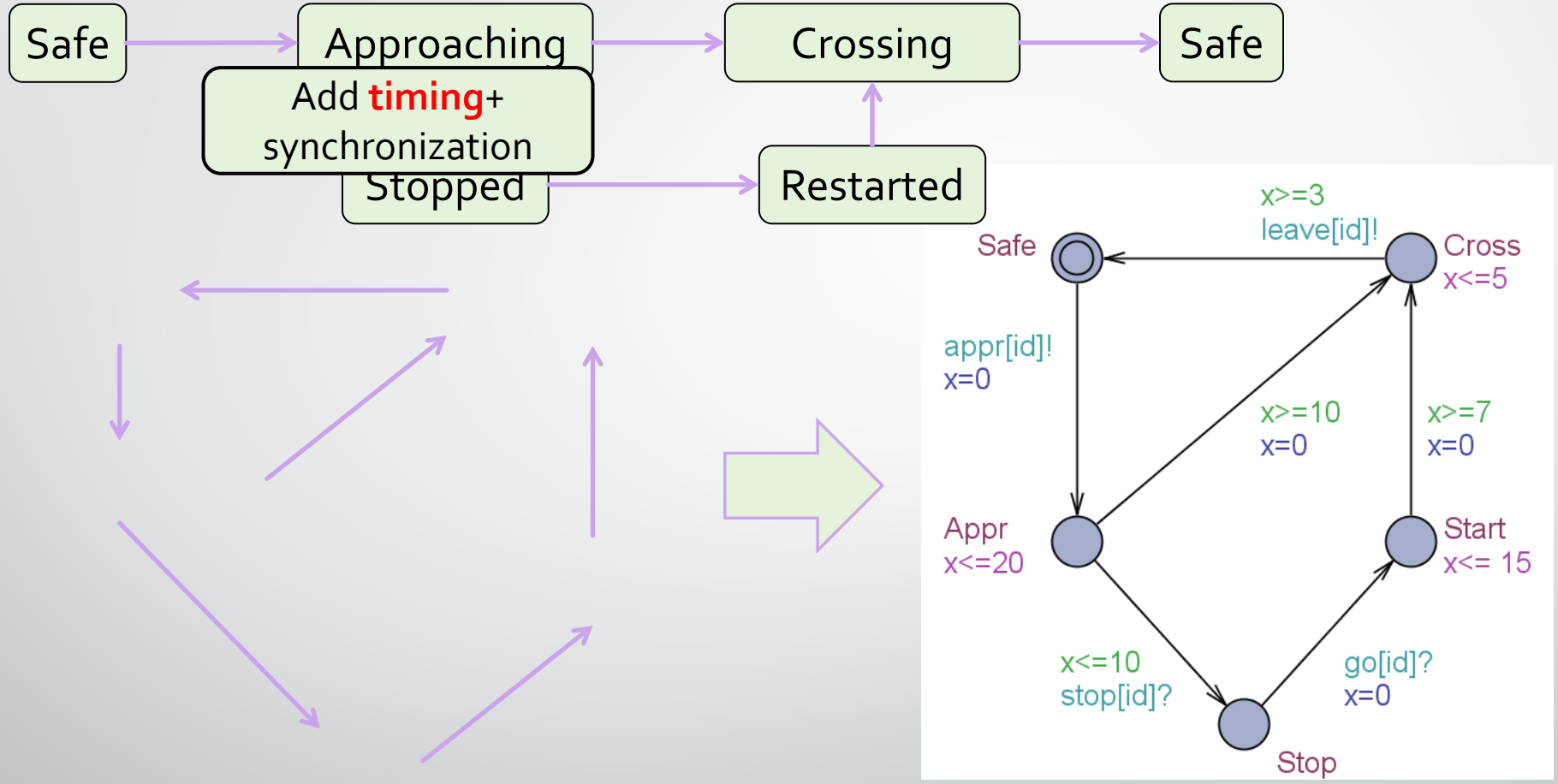


Stop the train while it still stoppable!

Train Crossing

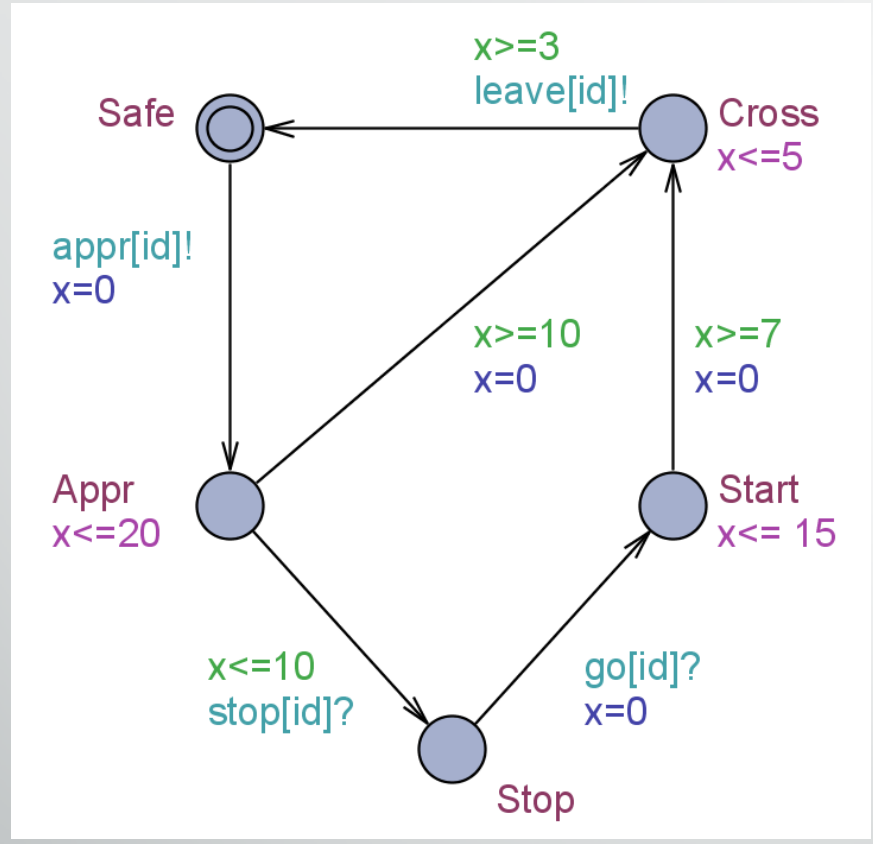


Train Crossing

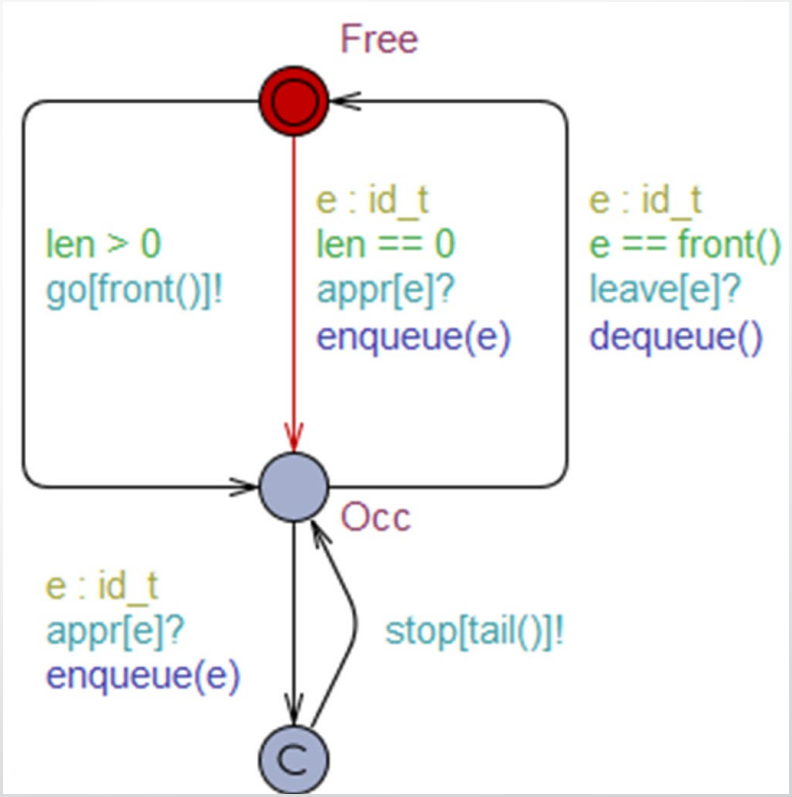



30%
15%
48%
Analysis

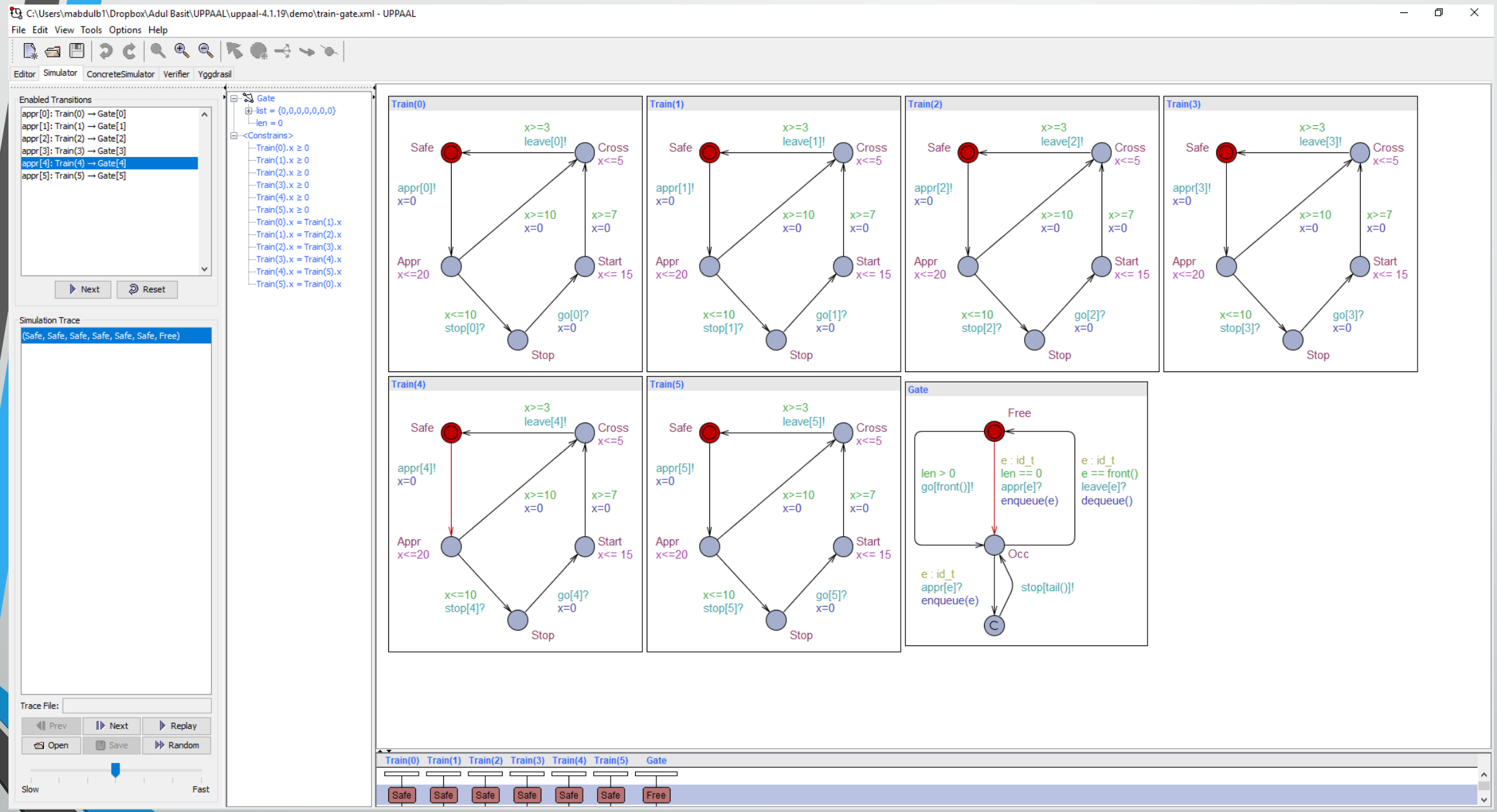
Example: Train Crossing



Train automata

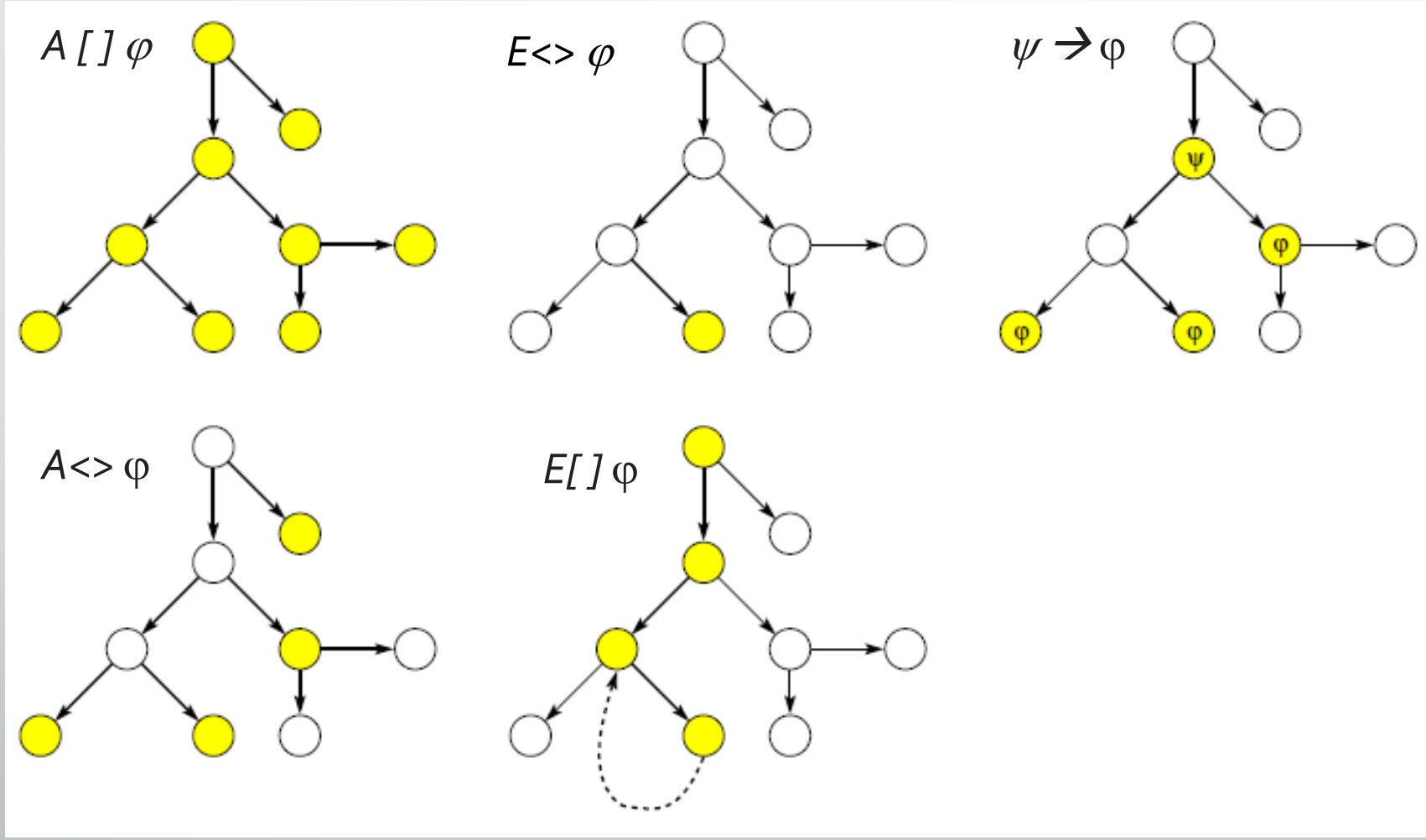


Gate automata

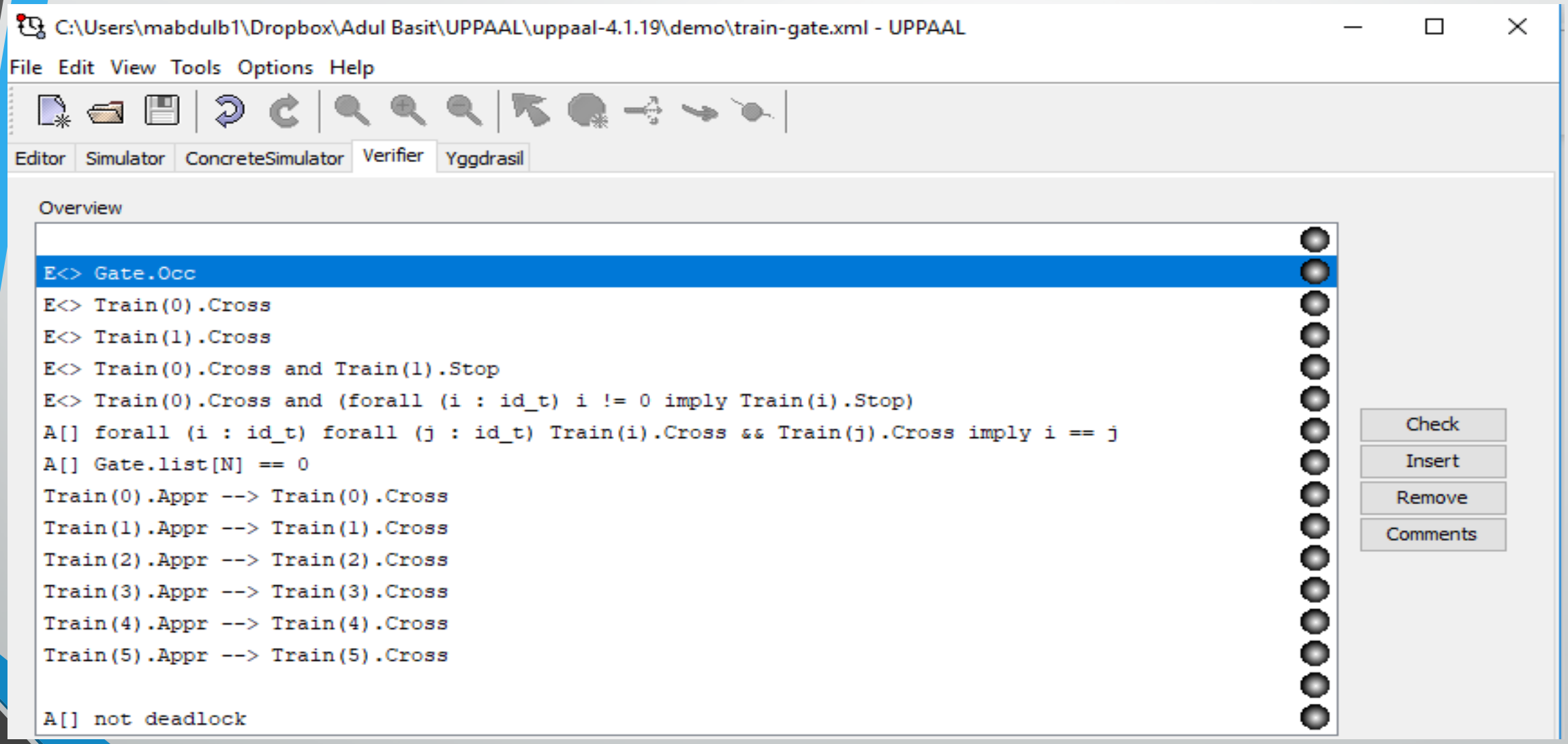


Example: Train-Gate.xml

Verifying Properties



Property Verification



Moving Target Defense - MTD

- Moving target defense allows the cyber systems to proactively defend against a wide-scale vector of sophisticated attacks by dynamically changing the system parameters and defense strategies in a timely and economical fashion.
- It can provide robust defense by deceiving attackers from reaching their goals, disrupting their plans via changing adversarial behaviors, and deterring them through prohibitively increasing the cost for attacks.
- MTD is distributed by nature since the processes or actions in MTD are executed in an interleaved manner.

Challenges

- If the parameters are not configured correctly then MTD does not remain **conflict-free** or system integrity will disrupt.
- Due to the dynamic nature of MTD, the configurable **parameters are dependent** on one another. Change in the value of one parameter affects the other parameters.
- MTD design must satisfy the **mutation constraints** and **design properties** which make sure that the system is **randomized**, **conflict-free**, **live**, and **progressive**.

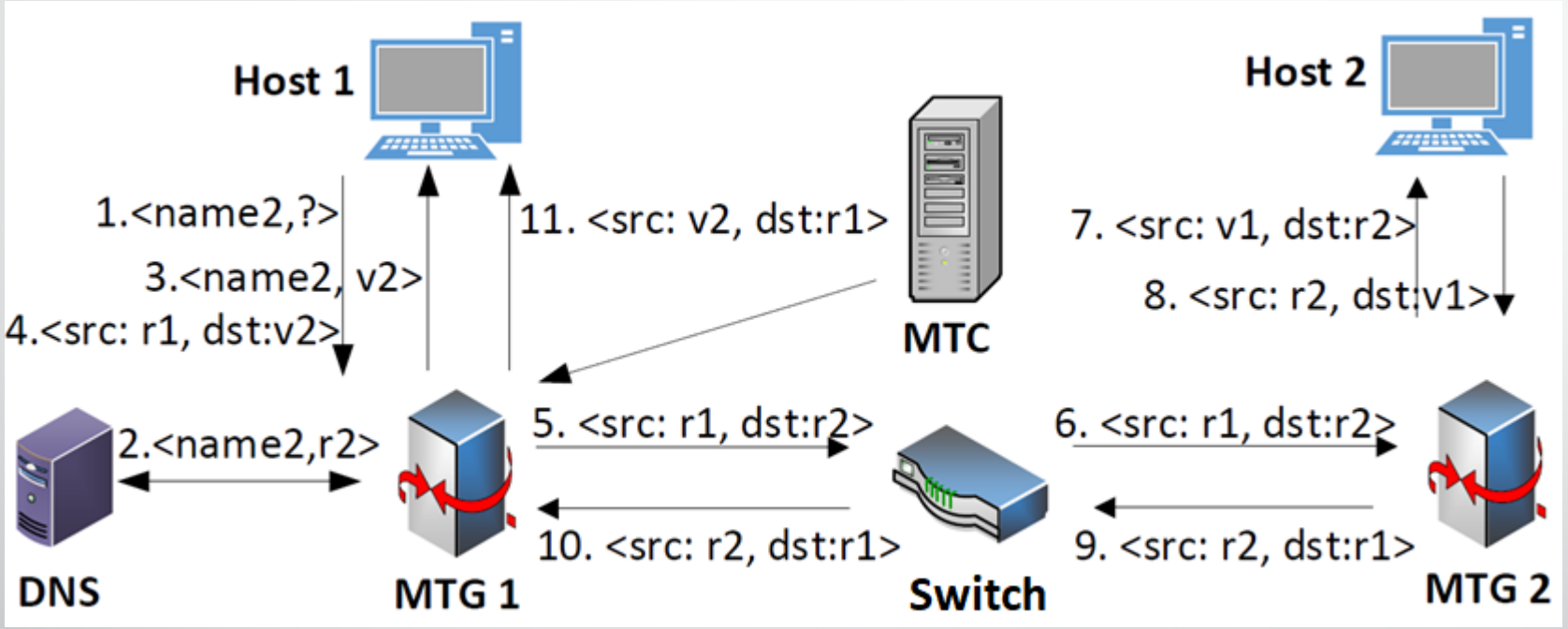
Techniques of MTD

- Random Host Mutation (RHM)
- Random Routing Mutation (RRM)
- Email Mutation (EM)

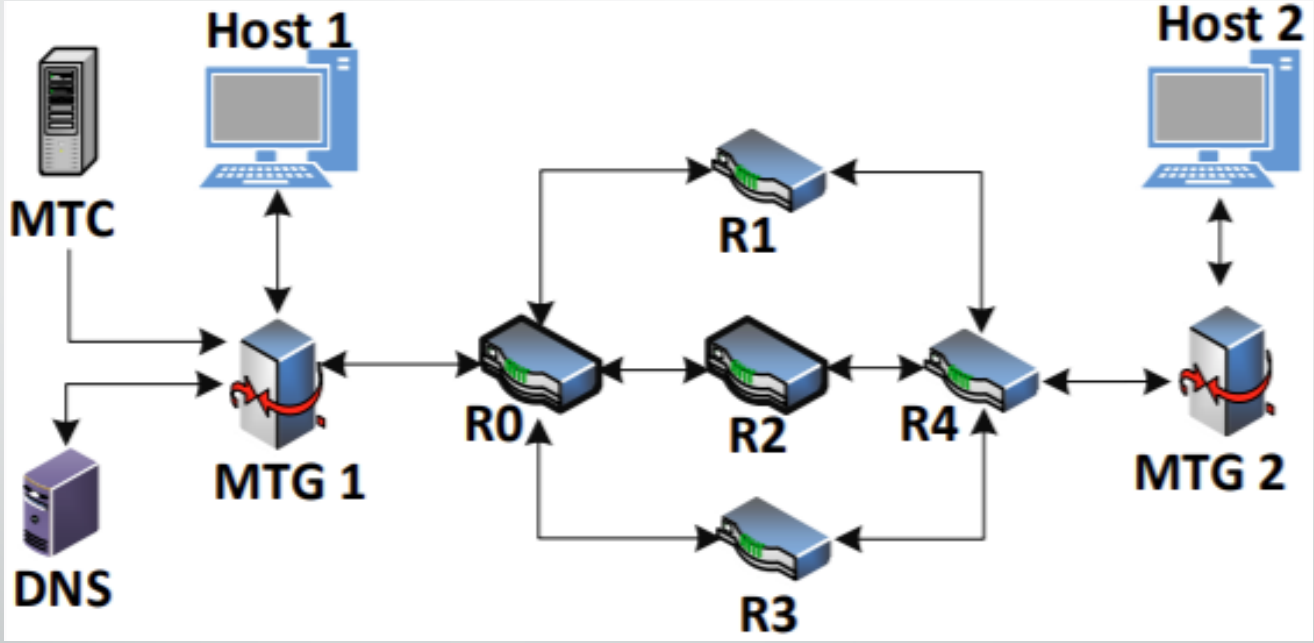
Random Host Mutation (RHM)

- **Funded by Army Research Office (ARO)**
- RHM is verified against the design correctness properties over numerous configurable and mutable parameters.
- A generalized model of RHM is prepared that ensures the scalability of verification of RHM.
- Simulation and verification ensure the design correctness and robustness of RHM.

Random Host Mutation Protocol (RHM)



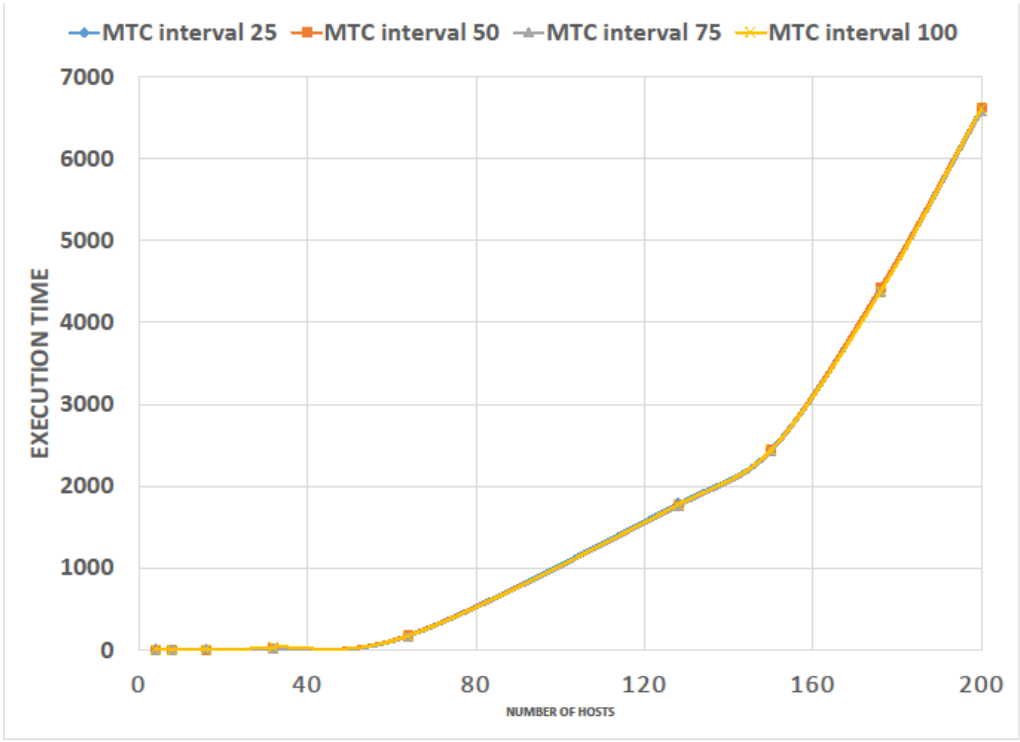
Random Routing Mutation (RRM)



Properties Verification

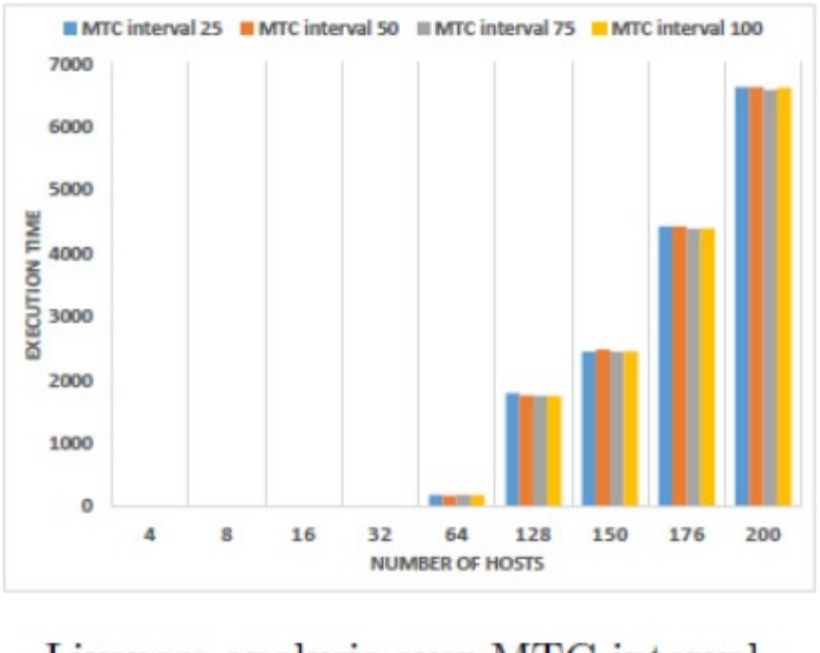
Property Verification	
Liveness	After a certain interval, MTC should mutate vIP that should be conflict-free and non-repetitive.
Fairness	New vIP for every mutating host should be broadcasted to MTG.
Deadlock-freeness	Deadlock can prevent MTG from mutation.

Evaluation



Reachability analysis over MTC intervals

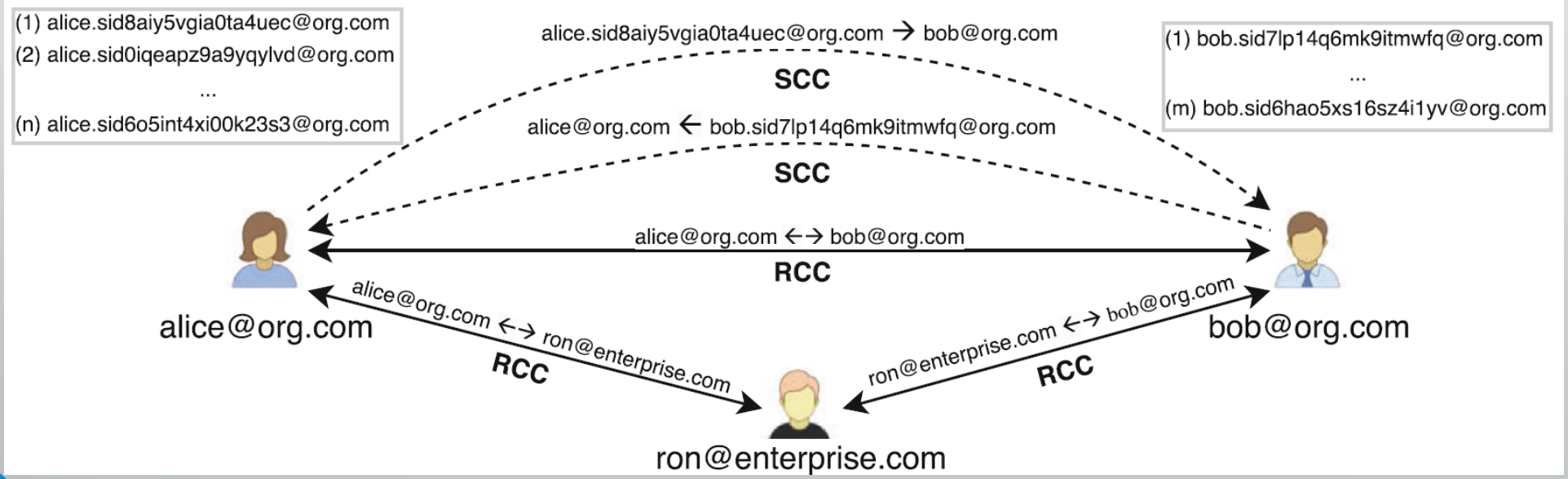
Evaluation



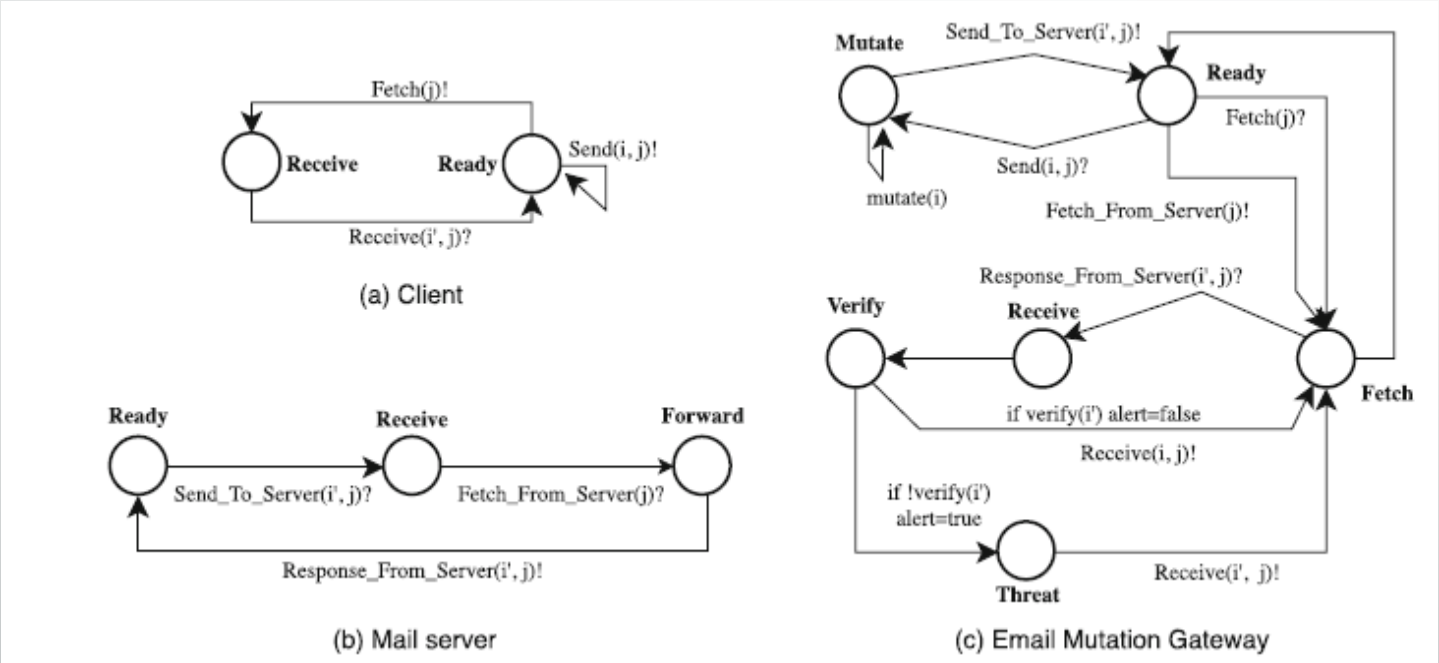
Liveness analysis over MTC interval

MTD - Email Mutation (EM)

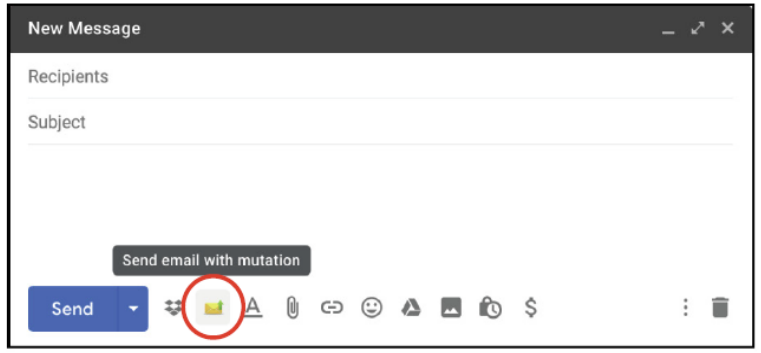
- **Funded by DARPA, ARO, ONR**
- EM is a novel protocol that provides proactive defense against highly stealthy lateral spear-phishing attacks.
- We verified that the EM protocol is valid and can be integrated with any existing email service provider.



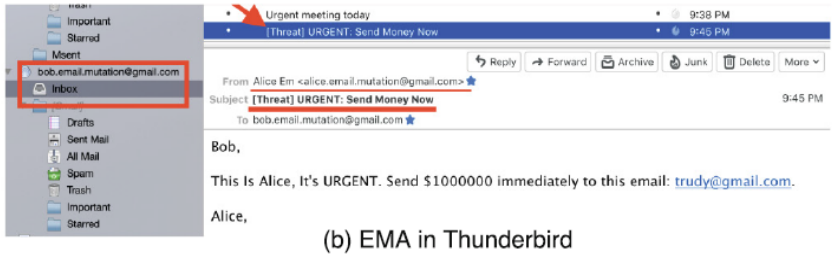
Formal Design Analysis



Browser Support



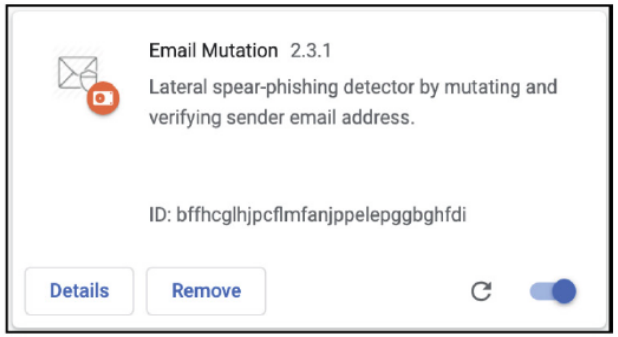
(a) Chrome Extension EMA adds new button for EM



(b) EMA in Thunderbird

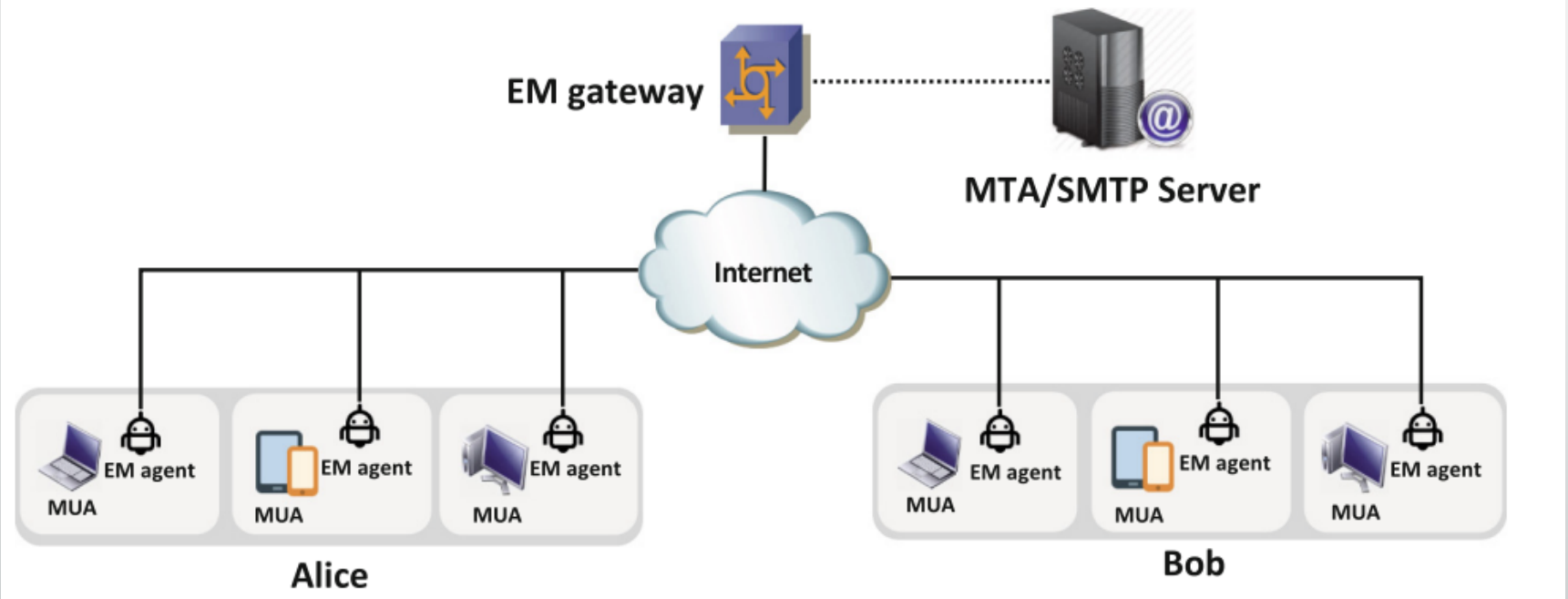


(c) EMA icon at Chrome Menu Bar



(d) EMA description in Chrome Extension Panel

EM - Architecture



Detection Results – Email Mutation

Metric	Data
Total attack emails	516,000
Lateral spear-phishing attack	153,207
Spoofing attack	145,291
EM engineering attack	201,437
Integrity attack	16,065
EM engineering attack missed	3
Integrity attack missed	167
L. spear-phishing detection	100%
Spoofing detection	100%
EM engineering detection	99.99%
EM engineering false negative	0.0015%
Integrity attack false negative	1.04%

Thanks

Q & A